KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

# 파이썬 기초 3
# in과
# 2차원 리스트

들어 있는지 묻고,
표 모양 데이터 다루기

```
"latte" in menu        → True
"tea" not in menu      → True
scores[1][2]           → 88
len(scores[0])         → 3
```

# 오늘의 순서

1

### in과 not in

리스트, 튜플, 문자열에서 들어 있는지 묻기, and와 or로 섞기, 한 겹만 보는 in

4–7쪽

2

### 2차원 리스트

[행][열]로 꺼내기, len으로 행 수와 열 수, for로 행과 열 다루기, 성적표, 좌석표, 함정과 오류

8–20쪽

3쪽은 노트북 준비, 21–22쪽은 막힐 때와 한 장 요약이에요.

들어가기

# 13에서 이어서 해요

- **hello-uv 노트북, 커널 .venv** — 09 · 25–27쪽
- **비교와 and, or, not** — 12 · 4파트
- **리스트, 튜플, [번호][번호]** — 13 · 2파트
- **for 문, += 로 합계 쌓기** — 13 · 2파트

기억이 안 나면 그 쪽을 먼저 다시 봐요.

### 노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

**이번 챕터의 코드도 모두 노트북 셀에 적어요.**

1. in과 not in

# in은 들어 있는지 True, False로 답해요

[1]
```python
menu = ["coffee", "latte", "cake"]
print("latte" in menu)
print("juice" in menu)
print("juice" not in menu)
print("latte" not in menu)
```

```
True
False
True
False
```

menu

| 0 | 1 | 2 |
|---|---|---|
| coffee | latte | cake |

| | | |
|---|---|---|
| "latte" in menu | True | latte가 있어요 |
| "juice" in menu | False | juice는 없어요 |
| "juice" not in menu | True | 없는 게 맞아요 |
| "latte" not in menu | False | 있으니까 틀렸어요 |

**not in은 in의 반대예요. 없으면 True, 있으면 False예요.**

1. in과 not in

# 튜플과 문자열에도 in을 써요

[2]
```python
order = ("coffee", 4500, 2)
print(4500 in order)
print(5000 in order)
print("4500" in order)
print("tea" not in order)
```

```
True
False
False
True
```

[3]
```python
word = "python"
print("th" in word)
print("pt" in word)
print("P" in word)
print("z" not in word)
```

```
True
False
False
True
```

`4500` `"4500"`

숫자 4500과 글자 "4500"은 다른 값이에요

`"th" in word`

붙어 있는 글자 조각을 찾아요. pt는 떨어져 있어서 False

`"P" in word`

대문자와 소문자를 구분해요. python에는 소문자 p만 있어요

**리스트, 튜플, 문자열 모두 in과 not in을 같은 모양으로 써요.**

1. in과 not in

# in의 결과는 and, or와 섞어 써요

[4]
```python
print(not "juice" in menu)
print("juice" not in menu)
has_both = "latte" in menu and "cake" in menu
print(has_both)
print("juice" in menu or "tea" in menu)
```

```
True
True
True
False
```

**결과를 변수에 담아 둘 수 있어요**

`has_both = "latte" in menu and "cake" in menu`

12의 4파트 is_adult = age >= 20 과 같은 모양이에요

`"juice" not in menu`

not "juice" in menu와 결과가 같아요. not in 쪽이 읽기 쉬워서 이렇게 써요

`A in menu and B in menu`

둘 다 있어야 True예요. latte와 cake가 모두 있어서 True

`A in menu or B in menu`

하나만 있어도 True예요. juice도 tea도 없어서 False

**나중에 배울 if 문은 이 True, False로 할 일을 골라요.**

1. in과 not in

# in은 한 겹만 들여다봐요

[5]
```python
orders = [("coffee", 4500, 2), ("latte", 5000, 1), ("cake", 6500, 3)]
print(("cake", 6500, 3) in orders)
print("cake" in orders)
print("cake" in orders[2])
```

```
True
False
True
```

**orders**

| 0 | 1 | 2 |
|---|---|---|
| ("coffee", 4500, 2) | ("latte", 5000, 1) | ("cake", 6500, 3) |

| | | |
|---|---|---|
| `("cake", 6500, 3) in orders` `True` | `"cake" in orders` `False` | `"cake" in orders[2]` `True` |
| 튜플 통째로는 찾아요 | 바깥에는 튜플 세 개만 있어요 | 한 겹 꺼낸 튜플 안에는 있어요 |

**안쪽까지 보려면 orders[2]처럼 한 겹 꺼낸 뒤 in을 써요.**

2. 2차원 리스트

# 리스트 안에 리스트를 넣으면 표가 돼요

[1]
```python
scores = [[90, 85, 77],
          [70, 95, 88],
          [60, 75, 99]]
print(scores)
print(len(scores))
print(len(scores[0]))
```

```
[[90, 85, 77], [70, 95, 88], [60, 75, 99]]
3
3
```

| 행＼열 | 0<br>kor | 1<br>eng | 2<br>math |
|---|---|---|---|
| 0 | 90 | 85 | 77 |
| 1 | 70 | 95 | 88 |
| 2 | 60 | 75 | 99 |

`len(scores)` **3** 행 수. 바깥 리스트에 든 줄의 개수

`len(scores[0])` **3** 열 수. 0행 안에 든 칸의 개수

**바깥 리스트 하나가 표, 안쪽 리스트 하나가 한 줄(행)이에요.**

2. 2차원 리스트

# [행][열] 순서로 꺼내요

[2]
```python
print(scores[1])
print(scores[1][2])
print(scores[0][0])
print(scores[-1][-1])
```

```
[70, 95, 88]
88
90
99
```

`scores[1]` `[70, 95, 88]`

| | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 90 | 85 | 77 |
| 1 | 70 | 95 | 88 |
| 2 | 60 | 75 | 99 |

1행 통째로

`scores[1][2]` `88`

| | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 90 | 85 | 77 |
| 1 | 70 | 95 | 88 |
| 2 | 60 | 75 | 99 |

1행의 2열

`scores[0][0]` `90`

| | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 90 | 85 | 77 |
| 1 | 70 | 95 | 88 |
| 2 | 60 | 75 | 99 |

맨 왼쪽 위

`scores[-1][-1]` `99`

| | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 90 | 85 | 77 |
| 1 | 70 | 95 | 88 |
| 2 | 60 | 75 | 99 |

마지막 행의 마지막 열

**앞 번호가 행(가로 줄), 뒤 번호가 열(세로 줄)이에요.**

# 칸 하나도, 줄 하나도 바꿀 수 있어요

[3]
```python
scores[0][1] = 100
print(scores[0])
scores[2] = [65, 80, 90]
print(scores)
```

```
[90, 100, 77]
[[90, 100, 77], [70, 95, 88], [65, 80, 90]]
```

`scores[0][1] = 100`

**번호 두 개** · 칸 하나를 바꿔요

`scores[2] = [65, 80, 90]`

**번호 하나** · 행을 통째로 바꿔요

첫째 줄 실행 뒤

| | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 90 | 100 | 77 |
| 1 | 70 | 95 | 88 |
| 2 | 60 | 75 | 99 |

셋째 줄 실행 뒤

| | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 90 | 100 | 77 |
| 1 | 70 | 95 | 88 |
| 2 | 65 | 80 | 90 |

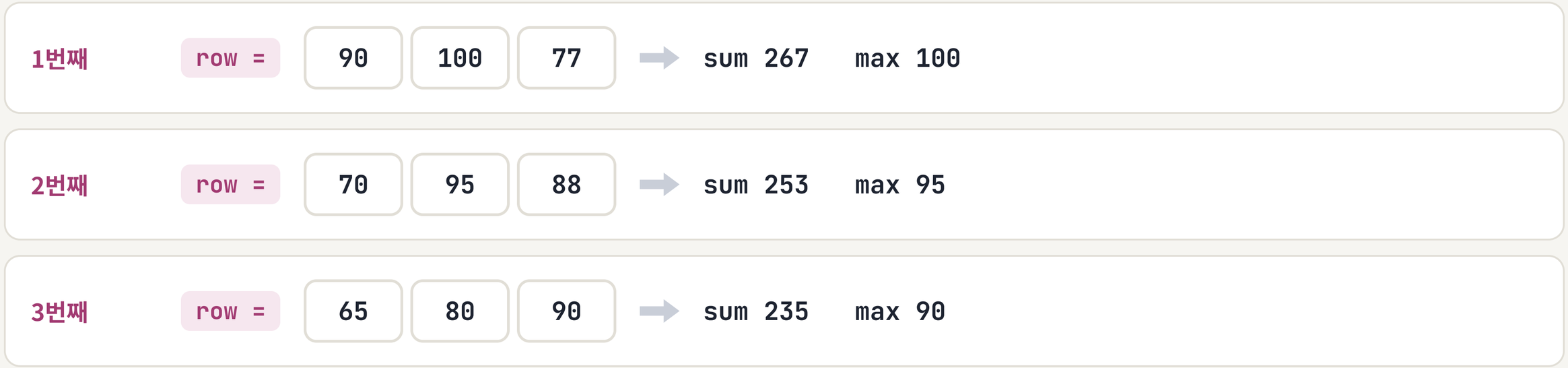
**13에서 본 리스트 바꾸기와 같아요. 번호를 몇 개 쓰는지만 달라요.**

2. 2차원 리스트

# for로 한 줄(행)씩 꺼내요

[4]
```python
for row in scores:
    print(row, "합계:", sum(row), "최고:", max(row))
```

```
[90, 100, 77] 합계: 267 최고: 100
[70, 95, 88] 합계: 253 최고: 95
[65, 80, 90] 합계: 235 최고: 90
```

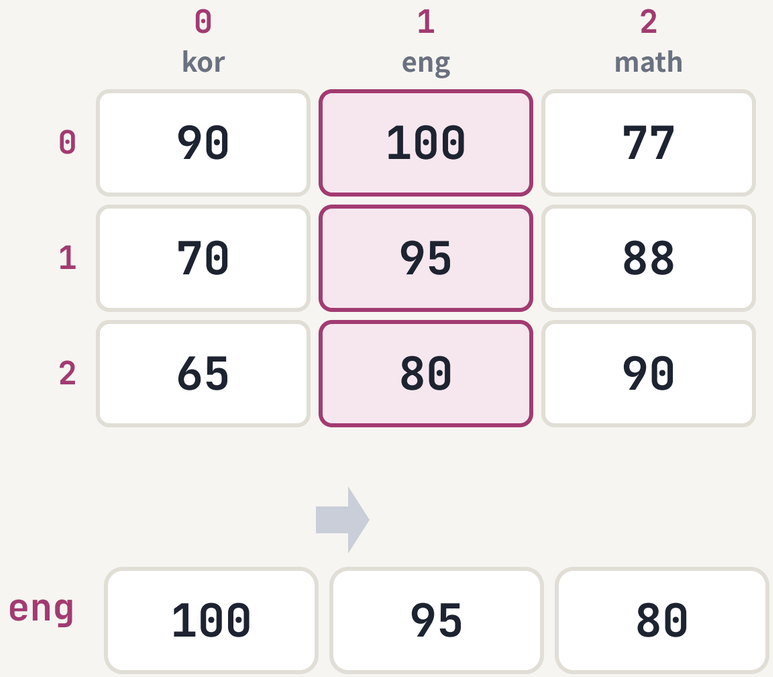



**row에는 숫자 하나가 아니라 한 행 리스트가 통째로 들어가요.**

# 열 하나는 for와 append로 모아요

[5]
```python
eng = []
for row in scores:
    eng.append(row[1])
print(eng)
print("영어 합계:", sum(eng))
print("영어 최저:", min(eng))
```

```
[100, 95, 80]
영어 합계: 275
영어 최저: 80
```




**행은 scores[1]로 바로 꺼내지만, 열은 행마다 [1]을 꺼내 모아요.**

2. 2차원 리스트

# 전체 합계는 행 합계를 쌓아요

[6]

```python
total = 0
count = 0
for row in scores:
    total += sum(row)
    count += len(row)
print("전체 합계:", total)
print("칸 수:", count)
print(f"평균: {total / count:.1f}")
```

Python

```
전체 합계: 755
칸 수: 9
평균: 83.9
```

| row | sum(row) | total | count |
|---|---|---|---|
| [90, 100, 77] | 267 | 267 | 3 |
| [70, 95, 88] | 253 | 520 | 6 |
| [65, 80, 90] | 235 | 755 | 9 |

**평균 = 전체 합계 ÷ 칸 수**

`f"{total / count:.1f}"`

13의 1파트 소수점 자리 맞추기로 한 자리만 보여 줘요

**total과 count는 for 앞에서 0으로 한 번만 시작해요.**

2. 2차원 리스트

# 이름까지 넣으면 성적표가 돼요

[7]
```python
students = [["kim", 90, 85, 77],
            ["lee", 70, 95, 88],
            ["park", 60, 75, 99]]
print(f"{'name':<6}{'kor':>5}{'eng':>5}{'math':>5}")
print("-" * 21)
for name, kor, eng, math in students:
    print(f"{name:<6}{kor:>5}{eng:>5}{math:>5}")
```

```
name    kor  eng math
---------------------
kim      90   85   77
lee      70   95   88
park     60   75   99
```

`for name, kor, eng, math in students:`

한 행의 값 네 개를 변수 네 개에 나눠 담아요

`for menu, price, qty in orders:`

13의 영수증 for와 같은 모양이에요

`{kor:>5}`

오른쪽 맞춤 다섯 칸. 13의 1파트 칸 수 맞추기

**행 안의 값 개수와 for 뒤의 변수 개수가 같아야 해요.**

# 2차원 리스트에서 in은 행 단위로 찾아요

[8]
```python
print(90 in students)
print(90 in students[0])
print(["lee", 70, 95, 88] in students)
for row in students:
    print(row[0], 99 in row)
```

```
False
True
True
kim False
lee False
park True
```

`90 in students` `False`

바깥에는 행 리스트 세 개만 있어요

`90 in students[0]` `True`

kim 행 안에는 90이 있어요

`["lee", 70, 95, 88] in students` `True`

행 통째로는 찾아요

`99 in row`

for로 행마다 물어요

**1파트의 orders와 같아요. in은 한 겹만 봐요.**

2. 2차원 리스트

# 자르기도 [행]과 [열]에 따로 써요

[9]
```python
print(scores[:2])
print(scores[1][:2])
print(scores[0][::-1])
```

```
[[90, 100, 77], [70, 95, 88]]
[70, 95]
[77, 100, 90]
```

**자르는 위치가 달라요**

scores[:2] 는 바깥 리스트를 잘라서 행을 가져와요.

scores[1][:2] 는 1행을 꺼낸 뒤 그 안의 칸을 잘라요.

[시작:끝:간격]은 13의 2파트에서 배운 그대로예요

`scores[:2]`

0행, 1행 두 줄

| | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 90 | 100 | 77 |
| 1 | 70 | 95 | 88 |
| 2 | 65 | 80 | 90 |

`scores[1][:2]`

1행 안의 0열, 1열

| | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 90 | 100 | 77 |
| 1 | 70 | 95 | 88 |
| 2 | 65 | 80 | 90 |

`scores[0][::-1]`

0행을 거꾸로

| | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 90 | 100 | 77 |
| 1 | 70 | 95 | 88 |
| 2 | 65 | 80 | 90 |

**번호 하나째 자리는 행, 둘째 자리는 열이에요. 자르기도 같아요.**

# 행은 append, 열은 for 안에서 append

[10]
```python
scores.append([80, 70, 60])
print(len(scores))
for row in scores:
    row.append(0)
for row in scores:
    print(row)
```

```
4
[90, 100, 77, 0]
[70, 95, 88, 0]
[65, 80, 90, 0]
[80, 70, 60, 0]
```

| | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | 90 | 100 | 77 | 0 |
| 1 | 70 | 95 | 88 | 0 |
| 2 | 65 | 80 | 90 | 0 |
| 3 | 80 | 70 | 60 | 0 |

`scores.append([80, 70, 60])` 새 행 하나. 바깥 리스트에 붙여요

`row.append(0)` 새 열. 행마다 하나씩 붙여요

**열을 더할 때는 행마다 하나씩 붙여야 해서 for를 써요.**

# 좌석표도 2차원 리스트예요

[11]
```python
seats = [["O", "O", "X", "O"],
         ["X", "O", "O", "O"],
         ["O", "X", "X", "O"]]
seats[0][1] = "X"
for row in seats:
    print(" ".join(row))
print(seats[2].count("X"))
print(len(seats) * len(seats[0]))
```

```
O X X O
X O O O
O X X O
2
12
```

앞 (화면)

| | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | O | X | X | O |
| 1 | X | O | O | O |
| 2 | O | X | X | O |

| | |
|---|---|
| `" ".join(row)` | 13의 join으로 한 행을 글자 한 줄로 |
| `seats[2].count("X")` | 2행의 찬 자리 2개 |
| `len(seats) * len(seats[0])` | 행 수 × 열 수 = 좌석 12개 |

**행과 열이 있는 데이터는 모두 이 모양으로 담아요.**

# [[0] * 3] * 3은 같은 행 하나를 세 번 담아요

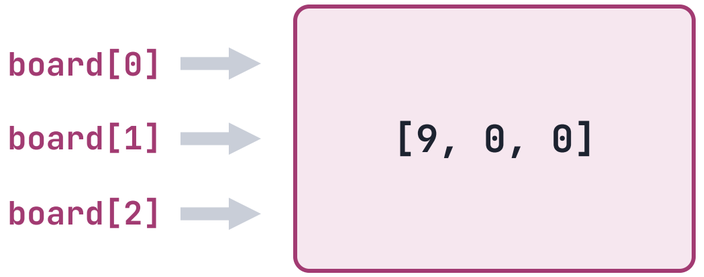
[12]
```python
board = [[0] * 3] * 3
board[0][0] = 9
print(board)
```

```
[[9, 0, 0], [9, 0, 0], [9, 0, 0]]
```

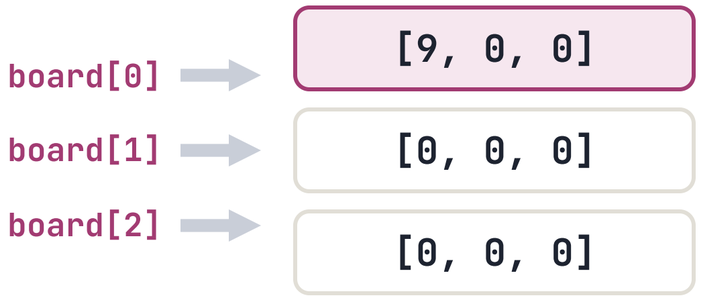
[13]
```python
board = [[0, 0, 0],
         [0, 0, 0],
         [0, 0, 0]]
board[0][0] = 9
print(board)
```

```
[[9, 0, 0], [0, 0, 0], [0, 0, 0]]
```

**[[0] * 3] * 3 · 행 하나를 셋이 같이 봐요**




**직접 적기 · 행이 세 개 따로 있어요**




**빈 표는 행을 하나씩 직접 적어서 만들어요.**

# 2차원 리스트에서 자주 나는 오류

[14]
```
scores[5][0]
```
Python

```
IndexError: list index out of range
```

행은 0~3번까지 네 개예요

[15]
```
scores[0][9]
```
Python

```
IndexError: list index out of range
```

0행 안의 칸은 0~3번까지예요

[16]
```
kor, eng, math = students[0]
```
Python

```
ValueError: too many values to unpack (expected 3)
```

행에는 이름까지 값이 네 개예요

**행 수는 len(scores), 열 수는 len(scores[0])로 확인해요.**

정리

# 막히면 오류의 마지막 줄을 봐요

1. **`TypeError: 'in <string>' requires string as left operand, not int`**
   문자열 안에 숫자가 있는지 물었어요
   → `"3" in word`

2. **`IndexError: list index out of range`**
   행이나 열 번호가 넘쳤어요
   → len()으로 확인

3. **`ValueError: too many values to unpack (expected 3)`**
   행의 값 개수와 변수 개수가 달라요
   → 변수를 값 수만큼

4. **`[[9, 0, 0], [9, 0, 0], [9, 0, 0]]`**
   칸 하나만 바꿨는데 모든 행이 바뀌었어요
   → 행을 직접 적기

5. **`90 in students → False`**
   in은 바깥 한 겹만 봐요
   → `90 in students[0]`

6. **`"P" in word → False`**
   대문자와 소문자가 달라요
   → `"p" in word`

**오류가 안 나도 결과가 이상하면 몇 겹을 보고 있는지 따져 봐요.**

# 한 장 요약

`"latte" in menu`

들어 있으면 True

`"tea" not in menu`

없으면 True

`"th" in word`

붙어 있는 글자 조각 찾기

`a in xs and b in xs`

둘 다 들어 있는지

`scores[1][2]`

1행 2열 칸 꺼내기

`len(scores[0])`

열 수 세기

`for row in scores:`

한 행씩 꺼내기

`col.append(row[1])`

열 하나 모으기

in은 한 겹만 봐요. 안쪽은 [번호]로 꺼낸 뒤 물어요

[행][열] 순서예요. 행 수는 len(scores), 열 수는 len(scores[0])